

WEAVE

A WHITEPAPER

PETETI DIVYA POORNA CHAND

SAI PRASANNA

SAI LOLITHA

ANUP CHAVAN

MIHIR DIVYANSH

B. PRANEETH

G. MEDHA SPANDANA



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Copyright © 2026 Peteti Divya Poorna Chand, Sai Prasanna, Sai Lolitha, Anup Chavan, Mihir Divyansh,
B. Praneeth, G. Medha Spandana

CS3423 COMPILERS, INDIAN INSTITUTE OF TECHNOLOGY HYDERABAD

First draft, September 2026

Contents

1	<i>Introduction to Weave</i>	7
1.1	Beginnings of the Weave Project	7
1.2	Design Goals of Weave	7
1.2.1	Small, Declarative, and Familiar	7
1.2.2	Checked Before Any Write	7
1.2.3	Structure-Aware and Link-Preserving	7
1.2.4	Effects in the Types	7
1.2.5	Deterministic and Reproducible	7
1.3	The Weave Base System	7
1.4	Weave and Agents—A New Approach to Editing Vaults	7
2	<i>Weave—Simple and Familiar</i>	9
2.1	Main Features of the Weave Language	9
2.1.1	The Vault Model: Notes, Properties, Sections, Links	9
2.1.2	Selections and Filters	9
2.1.3	Views: Tables and Lists	9
2.1.4	Documents: Composition from Sources	9
2.1.5	Migrations: Passes and Rules	9
2.1.6	Policies: <code>on_missing</code> and <code>on_conflict</code>	9
2.1.7	Null Handling and Diagnostics	9
2.2	Features Left Out on Purpose	9
2.2.1	No General Loops or Recursion	9
2.2.2	No String Replacement over Raw Text	9
2.2.3	No Writes Inside Views	9
2.2.4	No Implicit Coercions	9
2.2.5	No Partial Application of a Plan	9
2.3	Summary	9
3	<i>Weave is Structure-Aware</i>	11
3.1	Why Text Is Not Enough: <code>grep</code> , <code>sed</code> , and the Prose Problem	11

3.2	What Is a Vault Snapshot?	11
3.3	Basics of Structure	11
3.3.1	Notes and Frontmatter	11
3.3.2	Sections and Headings	11
3.3.3	Blockquotes and Body Nodes	11
3.3.4	Links and Their Resolution	11
3.3.5	Origins and Source Spans	11
3.3.6	Rewriting by Span, Not by Tree	11
3.4	Summary	11
4	<i>Checked, Atomic, and Robust</i>	13
4.1	Whole-Plan Checking	13
4.1.1	Overlapping Edits	13
4.1.2	Destination Collisions	13
4.1.3	Ambiguous References	13
4.2	All or Nothing: Staging and Application	13
4.3	Robust: Snapshot Hashes and the Recovery Journal	13
4.3.1	Compile-Time and Vault-Time Errors	13
4.4	Summary	13
5	<i>Rename Is Not Find-and-Replace</i>	15
5.1	The Link Invariant	15
5.1.1	The Broken-Link Problem	15
5.1.2	Resolving Before Rewriting	15
5.1.3	Supported Reference Forms	15
5.1.4	The Compatibility Boundary	15
5.2	Summary	15
6	<i>The Weave Compiler</i>	17
6.1	Two Frontends: the DSL and Markdown	17
6.2	The Type and Effect Checker	17
6.3	The Intermediate Representation	17
6.4	Passes and Their Legality Conditions	17
6.5	Optimisation: Shared Scans and Early Filtering	17
6.6	The Reference Interpreter as Oracle	17
6.7	Summary	17
7	<i>Queries and Composition</i>	19
7.1	Views over Metadata	19
7.2	Reaching into the Body	19
7.3	Assembling Documents with Provenance	19
7.4	Relationship to Dataview and Bases	19
7.5	Summary	19

8	<i>Performance and Comparisons</i>	21
8.1	Performance	21
8.2	Weave Compared: grep, Python, Linter, Dataview, Bases	21
8.3	A Major Benefit of Weave: Fearless Refactoring	21
8.4	Summary	21
9	<i>Weave for Agents</i>	23
9.1	The Evolution of Vault Editing	23
9.1.1	First-Generation Tools: Scripts and Regexes	23
9.1.2	Agents—A New Kind of User	23
9.1.3	The Essential Difference	23
9.1.4	A Plan Is Small and Reviewable	23
9.1.5	A Plan Cannot Be Half-Applied	23
9.1.6	Syntax Costs an Agent Nothing	23
9.2	Freedom to Restructure	23
9.3	Implementation Details	23
9.4	Safety	23
9.4.1	The First Layer—the Checker	23
9.4.2	The Next Layer—Preview and Apply	23
9.5	Weave for Agents—the Promise	23
10	<i>Further Reading</i>	25

1

Introduction to Weave

1.1 *Beginnings of the Weave Project*

1.2 *Design Goals of Weave*

1.2.1 *Small, Declarative, and Familiar*

1.2.2 *Checked Before Any Write*

1.2.3 *Structure-Aware and Link-Preserving*

1.2.4 *Effects in the Types*

1.2.5 *Deterministic and Reproducible*

1.3 *The Weave Base System*

1.4 *Weave and Agents—A New Approach to Editing Vaults*

2

Weave—Simple and Familiar

2.1 Main Features of the Weave Language

2.1.1 The Vault Model: Notes, Properties, Sections, Links

2.1.2 Selections and Filters

2.1.3 Views: Tables and Lists

2.1.4 Documents: Composition from Sources

2.1.5 Migrations: Passes and Rules

2.1.6 Policies: `on_missing` and `on_conflict`

2.1.7 Null Handling and Diagnostics

2.2 Features Left Out on Purpose

2.2.1 No General Loops or Recursion

2.2.2 No String Replacement over Raw Text

2.2.3 No Writes Inside Views

2.2.4 No Implicit Coercions

2.2.5 No Partial Application of a Plan

2.3 Summary

3

Weave is Structure-Aware

3.1 *Why Text Is Not Enough: grep, sed, and the Prose Problem*

3.2 *What Is a Vault Snapshot?*

3.3 *Basics of Structure*

3.3.1 *Notes and Frontmatter*

3.3.2 *Sections and Headings*

3.3.3 *Blockquotes and Body Nodes*

3.3.4 *Links and Their Resolution*

3.3.5 *Origins and Source Spans*

3.3.6 *Rewriting by Span, Not by Tree*

3.4 *Summary*

4

Checked, Atomic, and Robust

4.1 Whole-Plan Checking

4.1.1 Overlapping Edits

4.1.2 Destination Collisions

4.1.3 Ambiguous References

4.2 All or Nothing: Staging and Application

4.3 Robust: Snapshot Hashes and the Recovery Journal

4.3.1 Compile-Time and Vault-Time Errors

4.4 Summary

5

Rename Is Not Find-and-Replace

5.1 The Link Invariant

5.1.1 The Broken-Link Problem

5.1.2 Resolving Before Rewriting

5.1.3 Supported Reference Forms

5.1.4 The Compatibility Boundary

5.2 Summary

6

The Weave Compiler

6.1 *Two Frontends: the DSL and Markdown*

6.2 *The Type and Effect Checker*

6.3 *The Intermediate Representation*

6.4 *Passes and Their Legality Conditions*

6.5 *Optimisation: Shared Scans and Early Filtering*

6.6 *The Reference Interpreter as Oracle*

6.7 *Summary*

7

Queries and Composition

7.1 *Views over Metadata*

7.2 *Reaching into the Body*

7.3 *Assembling Documents with Provenance*

7.4 *Relationship to Dataview and Bases*

7.5 *Summary*

8

Performance and Comparisons

8.1 Performance

8.2 Weave Compared: grep, Python, Linter, Dataview, Bases

8.3 A Major Benefit of Weave: Fearless Refactoring

8.4 Summary

9

Weave for Agents

9.1 The Evolution of Vault Editing

9.1.1 First-Generation Tools: Scripts and Regexes

9.1.2 Agents—A New Kind of User

9.1.3 The Essential Difference

9.1.4 A Plan Is Small and Reviewable

9.1.5 A Plan Cannot Be Half-Applied

9.1.6 Syntax Costs an Agent Nothing

9.2 Freedom to Restructure

9.3 Implementation Details

9.4 Safety

9.4.1 The First Layer—the Checker

9.4.2 The Next Layer—Preview and Apply

9.5 Weave for Agents—the Promise

10

Further Reading

